



ARISTOTLE
UNIVERSITY
OF THESSALONIKI

Βαθιά Μάθηση και Ανάλυση Πολυμεσικών Δεδομένων

Βαθιά Ενισχυτική Μάθηση

Γεώργιος Κολοκοτρώνης

ΑΜ:143

Ημερομηνία: 07/06/2025

Εισαγωγή

Στην παρούσα εργασία, ανέπτυξα ένα μοντέλο βαθιάς ενισχυτικής μάθησης (Deep Reinforcement Learning – DRL) για την εκπαίδευση ενός έξυπνου πράκτορα που μαθαίνει να παίζει το κλασικό παιχνίδι Snake. Ο πράκτορας βασίζεται στον αλγόριθμο Deep Q-Network (DQN) και λαμβάνει αποφάσεις κίνησης με στόχο τη συλλογή τροφής και την αποφυγή εμποδίων, με απώτερο σκοπό τη μεγιστοποίηση της βαθμολογίας του. Η υλοποίηση έγινε εξ ολοκλήρου σε Python και αξιοποιεί τη βιβλιοθήκη PyTorch για την εκπαίδευση του νευρωνικού δικτύου. Το περιβάλλον προσομοίωσης περιλαμβάνει επιπλέον ένα στοιχείο: τον «μαγνήτη», ο οποίος ενεργοποιείται περιοδικά και επιτρέπει στον πράκτορα να συλλέγει τροφή από μικρή απόσταση, εισάγοντας έτσι ένα επιπλέον στρατηγικό στοιχείο στο gameplay. Μέσα από συνεχή αλληλεπίδραση με το περιβάλλον, ο πράκτορας κατάφερε να αναπτύξει αποτελεσματικές στρατηγικές για να επιβιώνει και να μεγιστοποιεί την απόδοσή του στο παιχνίδι.

Μεθοδολογία

Βιβλιοθήκες Python

Για την υλοποίηση της παρούσας εργασίας χρησιμοποιήθηκαν ορισμένες βασικές βιβλιοθήκες της Python, απαραίτητες για τη δημιουργία, εκπαίδευση και αξιολόγηση του πράκτορα βαθιάς ενισχυτικής μάθησης. Η βασικές ήταν :

- **NumPy**: Χρησιμοποιήθηκε για αποδοτικούς υπολογισμούς με πίνακες, ιδίως για την αναπαράσταση καταστάσεων και την επεξεργασία αριθμητικών δεδομένων του περιβάλλοντος.
- **PyTorch**: Αποτελεί τη βασική βιβλιοθήκη για τον ορισμό και την εκπαίδευση του νευρωνικού δικτύου Deep Q-Network (DQN). Επιτρέπει την εύκολη κατασκευή του μοντέλου, την εφαρμογή της μεθόδου backpropagation, και τη βελτιστοποίηση μέσω του Adam optimizer.

Περιβάλλον παιχνιδιού Snake

Για την εκπαίδευση του πράκτορα, υλοποιήθηκε ένα προσαρμοσμένο περιβάλλον τύπου OpenAI Gym που προσομοιώνει το κλασικό παιχνίδι Snake. Ο πράκτορας ελέγχει το φίδι, το οποίο κινείται σε ένα τετραγωνικό πλέγμα με σκοπό να συλλέγει τροφή και να αποφεύγει την πρόσκρουση στα τοιχώματα ή στο ίδιο του το σώμα. Η κατάσταση (state) που λαμβάνει ο πράκτορας περιλαμβάνει στοιχεία όπως την κατεύθυνση του φιδιού, τον σχετικό προσανατολισμό της τροφής, πιθανές συγκρούσεις μπροστά ή στα πλάγια, καθώς και κανονικοποιημένες συντεταγμένες. Το περιβάλλον είναι ενισχυμένο με έναν μηχανισμό «μαγνήτη», ο οποίος εμφανίζεται περιοδικά και επιτρέπει στον πράκτορα να συλλέξει τροφή αν πλησιάσει σε απόσταση ενός κελιού. Η λειτουργία του περιβάλλοντος περιλαμβάνει επίσης μηχανισμό ανταμοιβής (reward function), που ενισχύει θετικά τη συλλογή τροφής και τιμωρεί τις συγκρούσεις ή τις μη αποδοτικές κινήσεις, διευκολύνοντας την αποτελεσματική εκπαίδευση μέσω βαθιάς ενισχυτικής μάθησης.

Προετοιμασία Δεδομένων

Η προετοιμασία των δεδομένων εισόδου για το νευρωνικό δίκτυο γίνεται μέσω της συνάρτησης `get_state`. Σε κάθε χρονικό βήμα, ο πράκτορας λαμβάνει μια παρατήρηση της κατάστασης του περιβάλλοντος, η οποία περιλαμβάνει πληροφορίες σχετικά με πιθανούς κινδύνους μπροστά, αριστερά και δεξιά του

φιδιού, την τρέχουσα κατεύθυνση κίνησης, τη σχετική θέση της τροφής σε σχέση με το κεφάλι του φιδιού, τις κανονικοποιημένες συντεταγμένες του φιδιού και της τροφής, καθώς και το ποσοστό κατάληψης του πλέγματος από το σώμα του φιδιού. Όλες αυτές οι πληροφορίες συγκεντρώνονται σε ένα διάνυσμα 16 χαρακτηριστικών, το οποίο αποτελεί την είσοδο στο Deep Q-Network. Με αυτόν τον τρόπο, ο πράκτορας έχει επαρκές πλαίσιο πληροφόρησης ώστε να μάθει στρατηγικές επιβίωσης και βελτιστοποίησης της απόδοσής του.

```
def distance(self, a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def get_state(self):
    head = self.snake[0]
    dir_x, dir_y = self.direction
    left = (-dir_y, dir_x)
    right = (dir_y, -dir_x)

    def danger(dx, dy):
        nx, ny = head[0] + dx, head[1] + dy
        return not (0 <= nx < self.size and 0 <= ny < self.size) or (nx, ny) in self.snake

    state = [
        int(danger(dir_x, dir_y)),
        int(danger(left[0], left[1])),
        int(danger(right[0], right[1])),
        int(self.direction == (0, -1)),
        int(self.direction == (1, 0)),
        int(self.direction == (0, 1)),
        int(self.direction == (-1, 0)),
        int(self.food[0] < head[0]),
        int(self.food[0] > head[0]),
        int(self.food[1] < head[1]),
        int(self.food[1] > head[1]),
        head[0] / (self.size - 1),
        head[1] / (self.size - 1),
        self.food[0] / (self.size - 1),
        self.food[1] / (self.size - 1),
        len(self.snake) / (self.size * self.size)
    ]
    return np.array(state, dtype=float)
```

Το Μοντέλο

Το νευρωνικό δίκτυο που χρησιμοποιήθηκε για τον υπολογισμό της συνάρτησης είναι ένα πλήρως συνδεδεμένο (fully connected) feedforward δίκτυο, υλοποιημένο με τη βιβλιοθήκη PyTorch. Η αρχιτεκτονική του αποτελείται από

τρία διαδοχικά επίπεδα: Μια είσοδο με 16 χαρακτηριστικά (όπως ορίζονται από τη συνάρτηση `get_state`), δύο κρυφά επίπεδα με 256 νευρώνες το καθένα και ενεργοποίηση ReLU, και ένα τελικό επίπεδο εξόδου με 3 νευρώνες, που αντιστοιχούν στις τρεις δυνατές ενέργειες του πράκτορα (αριστερά, ευθεία, δεξιά). Η έξοδος του δικτύου αποτελεί το εκτιμώμενο Q-value για κάθε δυνατή ενέργεια στην τρέχουσα κατάσταση, και η ενέργεια με τη μέγιστη τιμή επιλέγεται κατά τη φάση. Η απλότητα της δομής του δικτύου επιτρέπει γρήγορη εκπαίδευση, ενώ η επαρκής χωρητικότητα των κρυφών επιπέδων εξασφαλίζει τη δυνατότητα εκμάθησης πολύπλοκων σχέσεων μεταξύ παρατήρησης και βέλτιστης δράσης.

```
class DQN(nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, 256)
        self.fc2 = nn.Linear(256, 256)
        self.fc3 = nn.Linear(256, output_dim)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)
```

Replay Buffer

Το Replay Buffer είναι μια μνήμη που αποθηκεύει τις εμπειρίες του πράκτορα από το παιχνίδι, δηλαδή την κατάσταση, την ενέργεια που πήρε, την ανταμοιβή, την επόμενη κατάσταση και αν το επεισόδιο τελείωσε. Αυτές οι εμπειρίες κρατούνται σε μια λίστα. Κατά την εκπαίδευση, το δίκτυο δεν μαθαίνει μόνο από την πιο πρόσφατη εμπειρία, αλλά επιλέγεται τυχαία ένα σύνολο εμπειριών (mini-batch) από τη μνήμη. Αυτό βοηθά στο να μην επηρεάζεται η μάθηση από μεμονωμένα γεγονότα και κάνει την εκπαίδευση πιο σταθερή και αποδοτική.

```

class ReplayBuffer:
    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)

    def push(self, *args):
        self.buffer.append(tuple(args))

    def sample(self, batch_size):
        batch = random.sample(self.buffer, batch_size)
        return map(np.array, zip(*batch))

    def __len__(self):
        return len(self.buffer)

```

Η κλάση ReplayBuffer διαχειρίζεται τη μνήμη εμπειριών του πράκτορα και περιλαμβάνει βασικές μεθόδους:

- **__init__**: Αρχικοποιεί μια deque (ουρά διπλής κατεύθυνσης) με προκαθορισμένη χωρητικότητα.
- **push**: Προσθέτει μία νέα εμπειρία στη μνήμη, με τα στοιχεία: κατάσταση, ενέργεια, ανταμοιβή, επόμενη κατάσταση και αν τελείωσε το επεισόδιο.
- **sample**: Επιλέγει τυχαία ένα σύνολο εμπειριών από το buffer και τα επιστρέφει ομαδοποιημένα για εκπαίδευση.
- **__len__**: Επιστρέφει το πλήθος των εμπειριών που έχουν αποθηκευτεί μέχρι στιγμής.

Εκπαίδευση

Η εκπαίδευση του πράκτορα πραγματοποιείται μέσα από τη συνάρτηση `train_snake`, η οποία είναι υπεύθυνη για την οργάνωση και εκτέλεση ολόκληρης της διαδικασίας μάθησης. Στην αρχή, ορίζονται οι βασικές υπερπαράμετροι του συστήματος, όπως ο ρυθμός μάθησης (learning rate), η τιμή προεξόφλησης μελλοντικών ανταμοιβών (για το discount factor γ), το μέγεθος παρτίδας για την εκπαίδευση (batch size), το όριο μεγέθους του replay buffer, καθώς και οι παράμετροι της πολιτικής διερεύνησης (epsilon, epsilon_min, epsilon_decay). Κατόπιν, δημιουργούνται τα δύο απαραίτητα νευρωνικά δίκτυα: το policy network, που εκπαιδεύεται κανονικά, και το target network, που αντιγράφει περιοδικά τα βάρη του πρώτου για να σταθεροποιήσει τη διαδικασία μάθησης. Η εκπαίδευση εκτελείται σε πολλαπλά επεισόδια. Για κάθε επεισόδιο, το περιβάλλον μηδενίζεται, και ο πράκτορας αλληλεπιδρά με αυτό έως ότου το παιχνίδι τελειώσει. Σε κάθε βήμα, επιλέγεται μια ενέργεια, η οποία εφαρμόζεται στο περιβάλλον και καταγράφεται η εμπειρία. Οι εμπειρίες αποθηκεύονται στον replay buffer. Όταν συγκεντρωθεί επαρκής αριθμός εμπειριών, δειγματίζεται ένα

υποσύνολο (mini-batch), το οποίο χρησιμοποιείται για να ενημερωθεί το policy network. Οι Q-τιμές υπολογίζονται, συγκρίνονται με τις αναμενόμενες τιμές (targets), και το δίκτυο εκπαιδεύεται με στόχο την ελαχιστοποίηση του σφάλματος (MSE Loss). Με την πάροδο των επεισοδίων, το epsilon μειώνεται σταδιακά, ώστε ο πράκτορας να βασίζεται περισσότερο στην ήδη εκπαιδευμένη πολιτική. Τέλος, αποθηκεύεται το εκπαιδευμένο μοντέλο και εμφανίζεται γράφημα με την πορεία της ανταμοιβής σε κάθε επεισόδιο.

```
def train_snake(num_episodes=50, render=False, use_magnet=True, return_logs=False):
    env = SnakeEnv(use_magnet=use_magnet)
    batch_size = 128
    gamma = 0.99
    lr = 1e-3
    buffer_capacity = 10000
    min_buffer_size = 1000
    epsilon = 1.0
    epsilon_min = 0.05
    epsilon_decay = 0.995
    target_update = 20

    obs_dim = env.observation_space.shape[0]
    n_actions = env.action_space.n
    policy_net = DQN(obs_dim, n_actions)
    target_net = DQN(obs_dim, n_actions)
    target_net.load_state_dict(policy_net.state_dict())
    optimizer = optim.Adam(policy_net.parameters(), lr=lr)
    replay_buffer = ReplayBuffer(buffer_capacity)

    all_rewards = []

    for episode in range(num_episodes):
        state = env.reset()
        total_reward = 0
        done = False

        while not done:
            if np.random.rand() < epsilon:
                action = random.randint(0, n_actions - 1)
            else:
                with torch.no_grad():
                    state_tensor = torch.FloatTensor(state).unsqueeze(0)
                    action = policy_net(state_tensor).argmax().item()

            next_state, reward, done, _ = env.step(action)
            replay_buffer.push(state, action, reward, next_state, done)
            state = next_state
            total_reward += reward

            if render:
                env.render()

            if len(replay_buffer) > min_buffer_size:
                s, a, r, ns, d = replay_buffer.sample(batch_size)
                s = torch.FloatTensor(s)
                a = torch.LongTensor(a)
                r = torch.FloatTensor(r)
                ns = torch.FloatTensor(ns)
                d = torch.FloatTensor(d)
                q_values = policy_net(s).gather(1, a.unsqueeze(1)).squeeze()
                next_q = target_net(ns).max(1)[0]
                expected_q = r + gamma * next_q * (1 - d)
                loss = nn.MSELoss()(q_values, expected_q.detach())
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

        all_rewards.append(total_reward)
```

Αξιολόγηση

```
def eval_snake(n_games=3, use_magnet=True):
    env = SnakeEnv(use_magnet=use_magnet)
    obs_dim = env.observation_space.shape[0]
    n_actions = env.action_space.n
    policy_net = DQN(obs_dim, n_actions)
    policy_net.load_state_dict(torch.load("best_snake_magnet_dqn.pth"))
    policy_net.eval()
    scores = []

    for game in range(n_games):
        state = env.reset()
        done = False
        while not done:
            env.render()
            with torch.no_grad():
                state_tensor = torch.FloatTensor(state).unsqueeze(0)
                action = policy_net(state_tensor).argmax().item()
            state, _, done, _ = env.step(action)
            print(f"Game {game+1} | Score: {env.score}")
            scores.append(env.score)

    avg_score = sum(scores) / len(scores)
    print(f"\nAverage Evaluation Score: {avg_score:.2f}")
```

Η αξιολόγηση του εκπαιδευμένου πράκτορα πραγματοποιείται μέσω της συνάρτησης `eval_snake`, η οποία φορτώνει το αποθηκευμένο μοντέλο και το εφαρμόζει σε νέο περιβάλλον χωρίς περαιτέρω μάθηση. Σε αυτή τη φάση δεν υπάρχει καθόλου τυχαιότητα στις αποφάσεις του. Ο πράκτορας ακολουθεί την προβλεπόμενη στρατηγική του νευρωνικού δικτύου και παίζει για προκαθορισμένο αριθμό παιχνιδιών (`n_games`). Σε κάθε επεισόδιο, καταγράφεται το σκορ και εμφανίζεται δυναμικά το περιβάλλον, επιτρέποντας στον χρήστη να παρακολουθήσει τη συμπεριφορά του πράκτορα. Μετά την ολοκλήρωση των παιχνιδιών, υπολογίζεται και εμφανίζεται ο μέσος όρος των επιδόσεων, καθώς και γράφημα που δείχνει τη βαθμολογία σε κάθε παιχνίδι. Η αξιολόγηση είναι σημαντική γιατί δείχνει πόσο καλά έμαθε ο πράκτορας να αποφεύγει εμπόδια και να μεγιστοποιεί την απόδοσή του, χρησιμοποιώντας μόνο την εμπειρία που απέκτησε κατά την εκπαίδευση.

Main Execution

Η εκτέλεση του κύριου προγράμματος γίνεται μέσω της ενότητας `if __name__ == "__main__":`, η οποία δίνει τη δυνατότητα στον χρήστη να επιλέξει αν θέλει να ξεκινήσει την εκπαίδευση ή την αξιολόγηση του πράκτορα. Ο χρήστης καλείται να εισάγει δύο επιλογές: πρώτον, αν θέλει να προχωρήσει σε εκπαίδευση (1) ή αξιολόγηση (2), και δεύτερον, αν επιθυμεί η λειτουργία του μαγνήτη να είναι ενεργή. Αν επιλεγεί η εκπαίδευση, εκτελείται η συνάρτηση `train_snake`, εκπαιδεύοντας τον πράκτορα για ορισμένο αριθμό επεισοδίων. Αντίστοιχα, αν επιλεγεί η αξιολόγηση, καλείται η `eval_snake`, για να φανεί στην πράξη η απόδοση του πράκτορα. Η δομή αυτή επιτρέπει την ευέλικτη χρήση του ίδιου αρχείου Python τόσο για πειραματισμούς όσο και για αναπαραγωγή των αποτελεσμάτων,

χωρίς την ανάγκη τροποποίησης του κώδικα.

```
if __name__ == "__main__":
    print("1. Train agent\n2. Evaluate agent")
    choice = input("Choose 1 or 2: ").strip()
    magnet_choice = input("Activate magnet? (y/n): ").strip().lower() == 'y'

    if choice == '1':
        rewards, avg = train_snake(num_episodes=1000, render=False, use_magnet=magnet_choice, return_logs=True)
        print(f"\nFinal Average Training Reward: {avg:.2f}")
    elif choice == '2':
        scores, avg = eval_snake(n_games=5, use_magnet=magnet_choice)
        print(f"\nFinal Average Test Score: {avg:.2f}")
```

env.render

Κατά την αξιολόγηση του πράκτορα, χρησιμοποιείται η μέθοδος `env.render()` για την οπτικοποίηση του περιβάλλοντος σε πραγματικό χρόνο. Μέσω αυτής, απεικονίζεται δυναμικά το πλέγμα του παιχνιδιού, η θέση του φιδιού, της τροφής και του μαγνήτη, καθώς και βασικές πληροφορίες όπως το σκορ και ο αριθμός βημάτων. Η δυνατότητα αυτή επιτρέπει την άμεση παρακολούθηση της συμπεριφοράς του πράκτορα, προσφέροντας ποιοτική αξιολόγηση της στρατηγικής του πέρα από τους αριθμητικούς δείκτες απόδοσης.

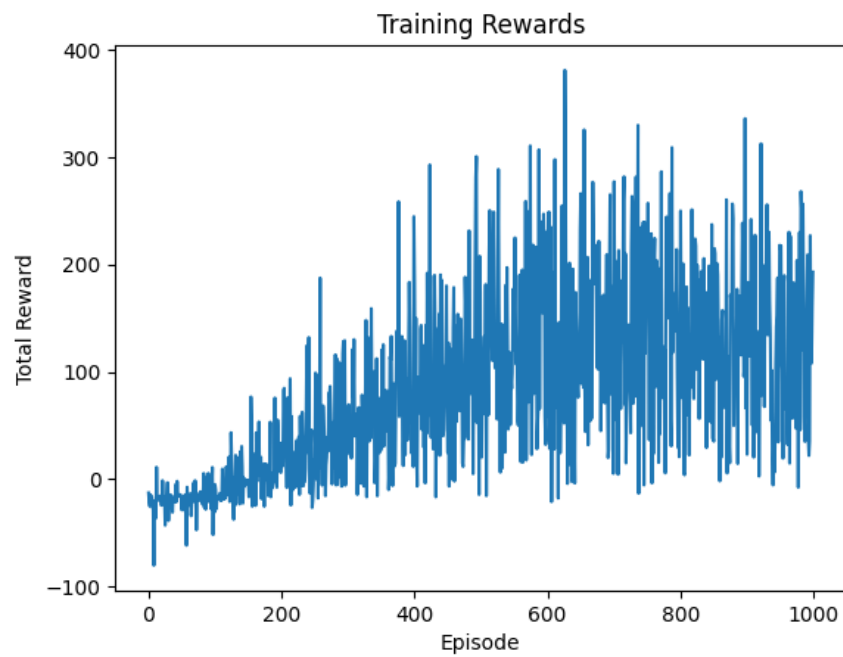
Αποτελέσματα

```
1. Train agent
2. Evaluate agent
Choose 1 or 2: 1
Activate magnet? (y/n): y

Episode 0 | Score: 1 | Total Reward: -12.70 | Epsilon: 0.995
Episode 1 | Score: 0 | Total Reward: -22.10 | Epsilon: 0.990
Episode 2 | Score: 0 | Total Reward: -25.40 | Epsilon: 0.985
Episode 3 | Score: 0 | Total Reward: -14.40 | Epsilon: 0.980
Episode 4 | Score: 0 | Total Reward: -18.80 | Epsilon: 0.975
Episode 5 | Score: 0 | Total Reward: -16.60 | Epsilon: 0.970
Episode 6 | Score: 0 | Total Reward: -25.40 | Epsilon: 0.966
Episode 7 | Score: 0 | Total Reward: -28.70 | Epsilon: 0.961
Episode 8 | Score: 0 | Total Reward: -80.40 | Epsilon: 0.956
Episode 9 | Score: 0 | Total Reward: -19.90 | Epsilon: 0.951
Episode 10 | Score: 0 | Total Reward: -24.30 | Epsilon: 0.946
Episode 11 | Score: 0 | Total Reward: -36.40 | Epsilon: 0.942
Episode 12 | Score: 2 | Total Reward: 11.20 | Epsilon: 0.937
Episode 13 | Score: 0 | Total Reward: -19.90 | Epsilon: 0.932
Episode 14 | Score: 0 | Total Reward: -18.80 | Epsilon: 0.928
Episode 15 | Score: 0 | Total Reward: -16.60 | Epsilon: 0.923
Episode 16 | Score: 0 | Total Reward: -16.60 | Epsilon: 0.918
```

Κατά τη διάρκεια της εκπαίδευσης, ο πράκτορας αρχικά εμφανίζει χαμηλή απόδοση, όπως φαίνεται στα πρώτα επεισόδια, με σκορ κοντά στο μηδέν και αρνητικά συνολικά rewards. Αυτό είναι αναμενόμενο καθώς ξεκινά με τυχαίες κινήσεις στο πλαίσιο της πολιτικής ερευνητικότητας (εψηλον-greedy). Καθώς προχωρά η εκπαίδευση και το epsilon μειώνεται, ο πράκτορας βασίζεται περισσότερο στο εκπαιδευμένο δίκτυο και σταδιακά βελτιώνει τη συμπεριφορά του. Από το επεισόδιο 500 και μετά, παρατηρείται σταθερή αύξηση στην απόδοση, με σκορ που φτάνουν τα 20–30 και συνολικά rewards άνω των 200. Η τελική μέση ανταμοιβή εκπαίδευσης είναι 83.89, επιβεβαιώνοντας ότι ο πράκτορας κατάφερε να μάθει αποδοτικές στρατηγικές για την επιβίωση και τη συλλογή τροφής. Η χρήση του μαγνήτη ενίσχυσε τις επιλογές και έδωσε

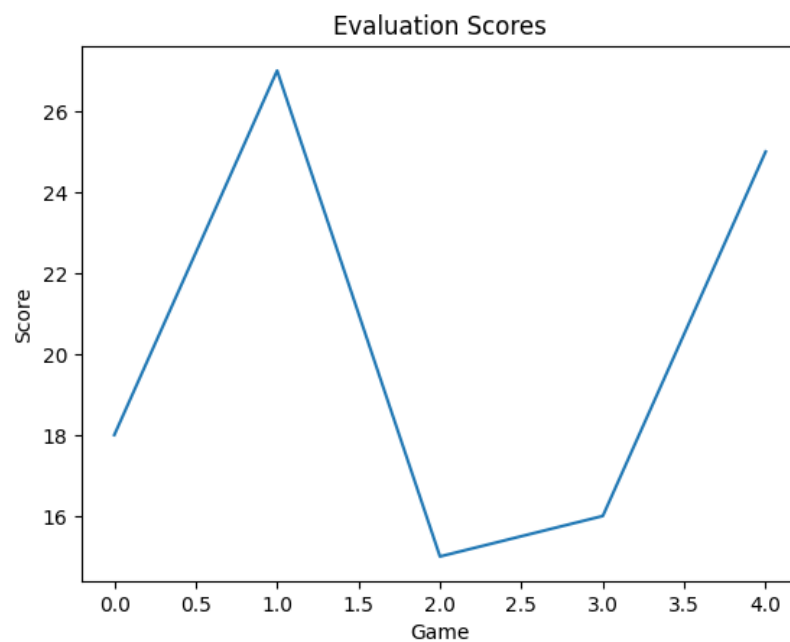
περαιτέρω δυνατότητες στρατηγικής.



TEST

Game 5 | Score: 25

Average Evaluation Score: 20.20



Final Average Test Score: 20.20

Στο στάδιο της αξιολόγησης, ο πράκτορας εφαρμόζει την εκπαιδευμένη στρατηγική του σε νέο περιβάλλον χωρίς καμία τυχαιότητα στις αποφάσεις του. Το παραπάνω γράφημα απεικονίζει το σκορ σε πέντε ανεξάρτητα παιχνίδια. Παρατηρείται ότι ο πράκτορας πετυχαίνει σταθερά υψηλές επιδόσεις, με σκορ

που κυμαίνονται από 15 έως 27 και μέσο όρο 20.20, γεγονός που επιβεβαιώνει την επιτυχημένη εκπαίδευση. Παρά τις μικρές αυξομειώσεις, τα αποτελέσματα δείχνουν ότι ο πράκτορας μπορεί να επιβιώνει, να βρίσκει τροφή και να αποφεύγει εμπόδια σε ποικιλία καταστάσεων. Η αξιολόγηση αυτή ενισχύει τη γενικευσιμότητα της στρατηγικής του πράκτορα και τη δυνατότητά του να προσαρμόζεται στο περιβάλλον.

Συμπεράσματα

Η εργασία αυτή πέτυχε τον στόχο της, εκπαιδεύοντας έναν πράκτορα με Deep Q-Network (DQN) να παίζει αποτελεσματικά το παιχνίδι Snake. Αρχικά η απόδοσή του ήταν χαμηλή, όμως καθώς μειωνόταν η τυχαιότητα (epsilon) και γινόταν εκπαίδευση από το replay buffer, η στρατηγική του βελτιώθηκε σημαντικά. Ο πράκτορας έμαθε να αποφεύγει εμπόδια, να εντοπίζει την τροφή και να αξιοποιεί(έστω και ελάχιστα)τον μαγνήτη για να μεγιστοποιεί την απόδοσή του. Η μέση ανταμοιβή αυξήθηκε σταθερά, και τα αποτελέσματα αξιολόγησης έδειξαν ότι η στρατηγική που έμαθε ήταν γενικεύσιμη και αποτελεσματική. Συνολικά, αποδείχθηκε ότι ακόμα και σε ένα απλό περιβάλλον, η ενισχυτική μάθηση μπορεί να οδηγήσει σε πολύπλοκη και αποδοτική συμπεριφορά.